

How does the quality of code changes made to meet business requirements differ between developers working with and without an AI coding agent?

Research proposal | ISYS90118

Introduction and Research Problem

Organisations adopting artificial intelligence (AI) tools need to know whether developers can deliver more work to the required standard. Miller et al. (2025) found that developers reported management expectations of faster delivery because they had access to AI. This raises a practical question about whether higher output also means more business changes that work correctly.

AI coding agents can edit files, run code and revise their work. When applying a change in an unfamiliar repository, developers still need to connect the business requirement with existing rules and dependencies. Agents may help them gather this context, but a finished change does not reveal all the assumptions or reasoning behind it.

Work across several repositories adds another demand: developers must switch between different business rules and code structures. Returning to a repository may reduce the effort needed to understand later changes. This study proposal will compare AI-assisted and manual work across four repositories within two hours, assessing delivered work alongside developers' accounts of their decisions.

The findings could help organisations to set workload expectations based on work that passes quality checks, rather than submission counts alone. The shared time limit represents a delivery constraint; the study will not isolate the effect of tighter deadlines.

Initial Literature Review

Time pressure and code quality before AI coding assistants

Research before widespread AI coding assistance linked time pressure with faster work but often poorer quality. Kuutila et al. (2020) reviewed 102 papers to assess how pressure affects software engineering. One manual-testing experiment found that time-restricted testers detected more defects per hour but fewer overall. Greater efficiency does not always mean a better result. However, findings varied across tasks and settings.

Fear of missing a deadline offers one explanation for this trade-off. Austin's (2001) model suggests that developers may take shortcuts despite caring about quality because they fear the consequences of late delivery. The model also suggests that extra time may not protect quality.

Kuutila et al. (2020) bring together evidence on the effects of time pressure, while Austin (2001) offers a possible explanation for developers' choices. His model does not directly test their behaviour.

AI support and developer understanding

AI support can help developers both write code and work out how to approach unfamiliar tasks. Barke et al. (2023) found that developers used Copilot to complete code they already knew how to write. When using an unfamiliar library, such as one for plotting data, they used its suggestions to decide what to do next.

Better task results, however, do not necessarily mean better understanding. Chen et al. (2025) compared Copilot with OpenHands, an AI agent that can edit and run code. They found that the OpenHands agent improved task correctness while reducing human effort, although participants raised concerns about understanding its outputs. J. H. Shen and Tamkin (2026) tested learning more directly: developers using a chat assistant to learn an unfamiliar Python library scored lower on a later knowledge test, with no significant average improvement in completion time.

These studies used different tools and tasks, so their results are not directly comparable. Still, they show why task correctness and developer learning need separate assessment.

Pressure during AI-assisted work

Time pressure can also affect how much work developers hand over to AI. Padmakumar et al. (2026) compared 40 developers building web applications with either one or four hours to finish. Developers given one hour had a 43% higher offloading score—an estimate of the share of work handled by AI—than those given four hours. However,

some developers with high scores still understood their code. Greater reliance therefore did not always mean less understanding.

Workplace expectations help explain why developers may face this pressure. Miller et al. (2025) interviewed 54 developers across 27 teams. Participants reported that management expected faster delivery because they had AI tools, while deadlines left little time to learn to use them effectively. The experiment shows how a shorter deadline can affect reliance; the interviews describe how expectations can restrict time for learning. Neither establishes whether developers correctly implement changing business requirements.

Requirement changes and later maintenance

Requirements shared after work begins may require changes to earlier code. Jiang et al. (2026) found that requirements introduced after an agent began editing were followed by more rewriting of its earlier code. This is relevant when developers receive a change or realise they omitted important context, but rewriting alone does not show that the result is worse.

AI-assisted code may be no harder to maintain. In Borg et al.'s (2026) study, developers first wrote code either with or without AI assistance. Other developers then extended that code without using AI. The researchers found no significant differences in completion time or code quality. However, they excluded solutions that failed the required tests, so the findings describe successful attempts rather than all attempts.

AI-assisted productivity and task outcomes

Productivity gains also depend on the work and setting. Becker et al. (2025) randomly allowed or disallowed AI on 246 tasks completed by 16 developers in repositories they knew well. Using early-2025 tools increased completion time by 19%, despite participants believing AI had helped them work faster. This finding concerns those tools and familiar projects, not all AI-assisted development.

In a later update, Becker et al. (2026) reported that reluctance to work without AI affected participant and task selection, making newer speed estimates unreliable. This highlights the need to consider who takes part and which tasks enter a comparison.

Balepur et al. (2026) studied 54 students using either an agent or a restricted chatbot to build an application. Agent users achieved better initial task results but lower comprehension scores. Both groups then extended their code with restricted chatbot support, so the study does not compare sustained agent use with fully manual work.

Conceptual Framework

Program comprehension will guide the study's exploration of developer reasoning. Pennington (1987) distinguishes understanding how a program runs from understanding the purpose it serves. Applying this distinction, the researcher will explore how participants connect a business need with the rules and code they choose to change.

The focus is on how that understanding develops through reading, agent interaction and return visits to a repository. Interviews will explore assumptions, alternatives and reasons for design choices, including plans for future changes that may not be clear from the code. These accounts will support interpretation of delivery outcomes, not test Pennington's original model.

Research Gap

The reviewed studies leave a specific question open. Becker et al. (2025) compared completion time on familiar repositories; Balepur et al. (2026) examined students building and extending one application; Padmakumar et al. (2026) compared deadlines during AI-assisted development. None directly answers how much acceptable work professional developers deliver across several initially unfamiliar repositories under a shared time limit.

The proposed study will examine that setting while allowing familiarity to grow through return visits. It will connect submission counts and quality results with developers' reasoning about requirements and code. The gap concerns delivery while moving between repositories, not AI productivity in general.

Research Aim and Question

This study aims to compare the amount and quality of business requirement changes delivered with and without an AI coding agent in two hours. Both groups will draw from the same task pool across four repositories. Interviews and observations will explore how developers gather context and make decisions alongside these delivery outcomes.

How does the quality of code changes made to meet business requirements differ between developers working with and without an AI coding agent?

Proposed Methodology and Methods

The study will use a mixed-method approach: a controlled comparison of delivery outcomes, supported by observation and open-ended interviews. Code checks will assess what the changes achieve. The qualitative evidence will explore how and why

developers made decisions, including reasoning that does not appear in the code. Combining these sources will connect the final work with the process behind it.

Comparative Research Design

Ten professional developers will need regular AI-tool experience, manual coding skills and no prior knowledge of the study repositories. Eligibility will depend on skills, not AI opinions. Before timing, all will practise both approaches on unrelated code. The researcher will record AI attitudes and randomly assign participants to two groups of five, balancing experience. Both groups will use the same tools and documentation, with AI disabled for the manual group and a fixed agent setup for the AI group.

Both groups will receive two hours and the same task pool, with no task cap. Tasks will rotate across four small repositories adapted from real projects. A pilot will check that tasks require broadly comparable effort and that the pool cannot be exhausted within the session. Two versus eight tasks is a planning estimate, not a quota or assumed gain. Assigned sequences will balance difficulty and repository order across groups.

Tasks, Procedure, and Measures

Each task will use an independent branch from a common baseline so an earlier error cannot block later tasks. Participants will retain their notes and, for AI users, agent conversations when returning to a repository.

Participants will mark a pull request as ready when they consider the task complete. Submission times will record claimed completion, not quality. At two hours, the researcher will save the latest versions of all submissions and unfinished work; later edits will not count.

Researcher-controlled end-to-end tests will check that new rules work and replaced rules no longer apply, following the distinction used in EvoCode-Bench (H. Shen et al., 2026). Unit tests will check established features. A quality engineer will test edge cases, and a reviewer familiar with the repository will assess submitted changes against a predefined checklist. Acceptance will require the agreed tests to pass and no blocking defects from either review. Reviewers will not see group labels where feasible.

The researcher will observe without coaching, noting files explored, assumptions, prompts, agent tool calls and corrections where relevant. With consent, screen recordings will support interviews after the session. Open-ended questions about selected successful, failed or unfinished work will explore how participants interpreted requirements, chose an approach and handled uncertainty. Participants will discuss their own code and decisions, not sit a knowledge test.

Data Analysis

The main productivity measure will be accepted tasks per developer in two hours. Submitted tasks, acceptance rates, defects and partial progress will distinguish output from work meeting the quality standard. Group summaries will show individual results, averages and ranges, considering task allocation. Acceptance rates will be undefined, not zero, for developers with no submissions.

The researcher will identify themes in observations and interview accounts, such as context gathering, assumptions and reasons for choices. These themes will inform accounts of individual tasks alongside their code and review results. Where accounts differ from observed actions or outcomes, the analysis will retain those differences and consider possible explanations. It will not convert interviews into scores or assume a defect proves poor understanding. Repository visits, time spent and technical failures will help interpret the findings.

Ethics and Data Management

Participants will give informed consent for observation, interviews and recordings and may withdraw without penalty. Recruitment will occur outside the researcher's workplace. Before data collection, the researcher will obtain required university ethics approval and explain AI-provider data handling and any limits on withdrawing data.

Code will remain in access-restricted private GitHub repositories. Recordings, transcripts and consent records will use university-approved secure storage, with access limited to authorised researchers and reviewers as needed. Participant codes will replace names in analysis, and identifying records will be stored separately. Retention and deletion will follow university requirements. Participants will receive the findings, and published reports will omit identifying details.

Expected Contribution

The study could show whether AI-assisted developers deliver more changes that meet the same quality standard within a fixed work period. If submissions increase without a similar increase in accepted work, submission counts would overstate the gain. If accepted work increases, that would support higher output expectations within the study's setting.

Developers' accounts could help explain the approaches behind these outcomes and identify where context gathering or support needs attention. Together, the findings could inform workload planning and onboarding across repositories. All outcomes will be reported, including limited gains, poorer quality or inconclusive differences.

Scope, Limitations, and Feasibility

Preparation will require removing confidential material from adapted repositories, piloting tasks and tests, recruiting participants and arranging quality review. Sessions require 20 developer-hours, plus interviews and assessment. Independent task branches limit the effect of earlier defects but do not reproduce the build-up of changes in a live project.

With five participants per group, individual experience may strongly affect results. Findings will offer exploratory estimates, not proof of a general three- to fourfold productivity gain. Small repositories, one AI setup and recruitment of willing regular AI users limit wider application. Faster participants may revisit repositories more often, affecting familiarity. Observation may affect behaviour, and interviews rely on recall. The study will not assess long-term maintenance or justify workforce reductions.

References

- Miller, C., Choudhuri, R., Ulloa, M., Haniyur, S., DeLine, R., Storey, M.-A., Murphy-Hill, E., Bird, C., & Butler, J. L. (2025). “*Maybe we need some more examples:*” *Individual and team drivers of developer GenAI tool use* [Preprint]. arXiv. <https://arxiv.org/abs/2507.21280>
- Kuuttila, M., Mäntylä, M., Farooq, U., & Claes, M. (2020). Time pressure in software engineering: A systematic review. *Information and Software Technology*, 121, Article 106257. <https://doi.org/10.1016/j.infsof.2020.106257>
- Austin, R. D. (2001). The effects of time pressure on quality in software development: An agency model. *Information Systems Research*, 12(2), 195–207. <https://doi.org/10.1287/isre.12.2.195.9699>
- Barke, S., James, M. B., & Polikarpova, N. (2023). Grounded Copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1), Article 78. <https://doi.org/10.1145/3586030>
- Chen, V., Talwalkar, A., Brennan, R., & Neubig, G. (2025). *Code with me or for me? How increasing AI automation transforms developer workflows* [Preprint]. arXiv. <https://arxiv.org/abs/2507.08149>
- Shen, J. H., & Tamkin, A. (2026). *How AI impacts skill formation* [Preprint]. arXiv. <https://arxiv.org/abs/2601.20245>
- Padmakumar, V., Ibrahim, L., Wang, Z. Z., Wang, J., Liao, Q. V., & Yang, D. (2026). *Offloading score: Measuring AI reliance through counterfactual workflows* [Preprint]. arXiv. <https://arxiv.org/abs/2605.29392>

- Jiang, B., Cheng, H., Fu, Y., Koziolok, A., Li, J., & Zhang, W. (2026). *Requirements after the first edit: Mining late requirement emergence and rework in real-world coding-agent sessions* [Preprint]. arXiv. <https://arxiv.org/abs/2609.03028>
- Borg, M., Hewett, D., Hagatulah, N., Couderc, N., Söderberg, E., Graham, D., Kini, U., & Farley, D. (2026). Echoes of AI: Investigating the downstream effects of AI assistants on software maintainability. *Empirical Software Engineering*, 31, Article 161. <https://doi.org/10.1007/s10664-026-10889-1>
- Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). *Measuring the impact of early-2025 AI on experienced open-source developer productivity* [Preprint]. arXiv. <https://arxiv.org/abs/2507.09089>
- Becker, J., Rush, N., Cunningham, T., Rein, D., & Mahamud, K. (2026, February 24). *We are changing our developer productivity experiment design*. METR. <https://metr.org/blog/2026-02-24-uplift-update/>
- Balepur, N., Baumler, C., Chen, V., Choi, E., Rudinger, R., & Boyd-Graber, J. (2026). *(Im)Paired programming: Coding agents improve productivity but harm understanding* [Preprint]. arXiv. <https://arxiv.org/abs/2607.26375>
- Pennington, N. (1987). Stimulus structures and mental representations in expert comprehension of computer programs. *Cognitive Psychology*, 19(3), 295–341. [https://doi.org/10.1016/0010-0285\(87\)90007-7](https://doi.org/10.1016/0010-0285(87)90007-7)
- Shen, H., Chen, X., Xu, W., Ma, Y., Chen, L., & Li, K. (2026). *EvoCode-Bench: Evaluating coding agents in multi-turn iterative interactions* [Preprint]. arXiv. <https://arxiv.org/abs/2605.24110>

Appendix A: RIOT Certificate

View Course

31/8/2026, 12:57 pm



← Research Integrity Online Training: RIOT

Research

Copy Link

Unenrol

Gurdeep Bajwa

Home

Profile

Learning

My Learning

Course Catalogue



Course Information

COMPLETED

Duration	2h
Enrolment date	31 Aug 2026
Start date	31 Aug 2026
Complete date	31 Aug 2026
Enrolment method	Self ⓘ

Powered by ELMO Software

<

Research Integrity Online Training helps to establish a common foundation and a shared understanding of responsible research requirements and practices at the University of Melbourne.

The course is [mandatory for graduate researchers](#). It is recommended for all staff, honorary appointments and visiting researchers who conduct or support research.

By completing RIOT participants will build their understanding of key research integrity concepts, requirements and resources to support the responsible research conduct at the University and gain confidence in applying these to their work.

Users who require an alternate mode of access can submit a request for assistance via [Staff Services](#) or [Student Services](#). For other queries or to request help [log a ServiceNow request for RIOT support](#).

Learning objectives

By the end of this course, you will be able to:

- Understand responsibilities under the [Australian Code for the Responsible Conduct of Research](#) and associated policies and procedures at the University of Melbourne
- Know how to apply the principles of research integrity to your research and how to seek help when you need it
- Have considered examples and literature
- Have reflected on your opinions about research integrity issues
- Be equipped with resources and tools to continue the conversation about research integrity with your colleagues.

Important information for graduate researchers

You will be required to provide a RIOT certificate of completion during the onboarding process.

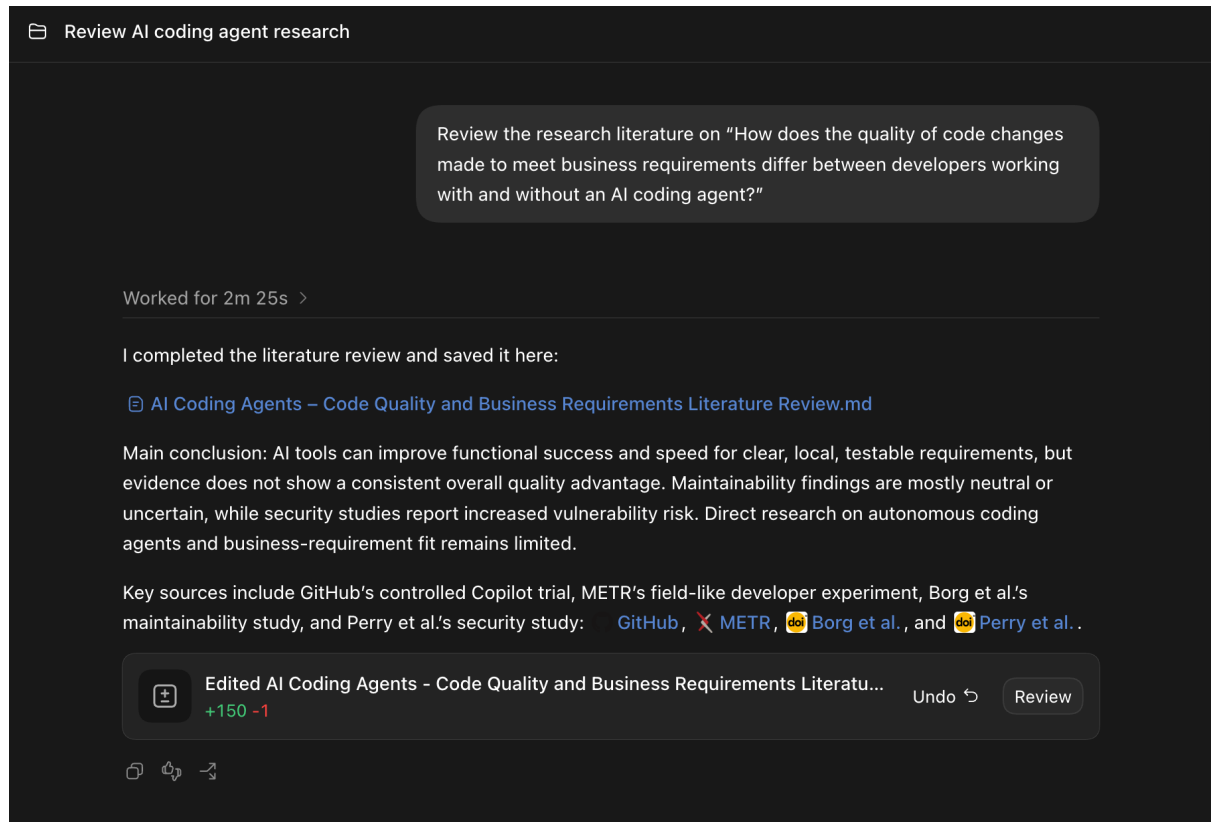
?

https://uomtrainme.almotalent.com.au/learning/view-course-enrolment/1466597

Page 1 of 1

Appendix B: ChatGPT Literature Review and Comparison

Prompt and AI response summary



The submission ZIP includes the full AI-generated literature review in “AI Coding Agents - Code Quality and Business Requirements Literature Review.md”. The screenshot above records the prompt and response summary.

Comparison

The AI review separates correctness, maintainability and security, which helps explain why code quality needs several checks. However, its overall answer suggests that AI performs worse as requirements become more ambiguous without presenting a direct test of that claim. My review focuses more closely on delivery pressure, developer understanding and requirements introduced during development. It includes agent studies that the AI review omits and explains how their settings differ from my proposed study. The AI output provides useful ways to assess quality, but my review better explains why understanding business needs and existing code matters when developers apply several changes.